PBCST304

OBJECT ORIENTED PROGRAMMING

MODULE - 2

Prepared by: Aswathy.T.K

Polymorphism :-

Method Overloading, Using Objects as Parameters, Returning Objects, Recursion.

Static Members, Final Variables, Inner Classes.

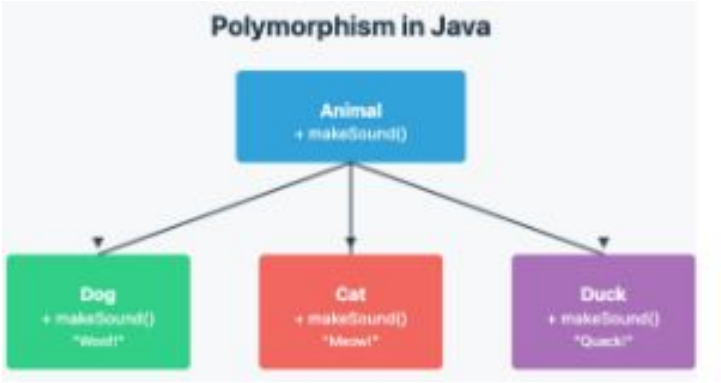
Inheritance - Super Class, Sub Class, Types of Inheritance, The ***super*** keyword, protected Members, Calling Order of Constructors.

Method Overriding, Dynamic Method Dispatch, Using ***final*** with Inheritance.

POLYMORPHISM

- The word polymorphism means "many forms."
- Polymorphism is a feature that **allows one interface to be used for a general class of actions.**
    - ie, “**one interface, multiple methods**”
- Polymorphism in Java is an Object-Oriented Programming (OOP) concept that **allows the same method name to perform different tasks depending on the object that invokes it.**
- Eg 1:
    - Consider an individual who plays distinct roles in their life, such as a teacher in school, a mother their kids, a daughter for their parents etc.,
    - Based on the above scenario, depending on the behaviour, the individual’s role will be different.
    - The roles are to be decided based on the object’s call.

Eg 2:



- There is a superclass: Animal
    - makeSound() is a method.
- There are 3 sub classes:
    - Dog->makeSound() -> “Bark”
    - Cat->makeSound() -> “Meow”
    - Duck>makeSound() -> “Quack”
- All three classes have the same method makeSound(), but each provides it’s own implementation.
- Here, **the same method call animal.makeSound() can produce different results depending on the actual object**. ===> POLYMORPHISM

- There are two types of polymorphism in Java:
  - **Compile-Time Polymorphism (Static polymorphism)**
    - Method Overloading
  - **Run-time Polymorphism (Dynamic polymorphism)**
    - Method Overriding
    - Dynamic method dispatch

Method Overloading: (Compile-Time Polymorphism)

- Method overloading is one of the ways that Java supports polymorphism.
- Method Overloading occurs when **multiple methods have the same name but different parameter lists**.
- The **compiler decides which method to call.**
- Method overloading supports polymorphism because it is one way that Java implements the “one interface, multiple methods” paradigm.

- **When an overloaded method is invoked, Java uses** the **type and/or number of arguments** as its guide **to determine which version of the overloaded method to actually call.**
- Thus, **overloaded methods must differ in the type and/or number of their parameters.**
- While overloaded methods may have different return types, the return type alone is insufficient to distinguish two versions of a method.
- **When an overloaded method is called, Java looks for a match between the arguments used to call the method and the method's parameters.**

Eg1:

```
class Calculator {
    int add(int a, int b)
        {
        return a + b;
        }
    double add(double a, double b) {
        return a + b; }
                    }
public class main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(10, 20));      // Calls int version
        System.out.println(c.add(5.5, 4.5));    // Calls double version
    }
}
```

OUTPUT:

30

10.0

- In the method call add(10, 20),
  - Arguments are 10 and 20.
  - The compiler checks all the overloaded versions of the add() method and selects the one whose parameter types best match the types of the arguments.
  - Since 10 and 20 are of type int, the compiler invokes the method: int add(int a, int b)

Advantages of Method Overloading in Java

- **Improves readability**
  - The same method name can be used for similar operations with different inputs.
- **Increases code reusability**
  - We can perform related operations using one common method name instead of creating different method names.
- **Supports compile-time polymorphism**
  - The compiler determines which overloaded method to call based on the number, type, or order of arguments.
- **Makes programs easier to understand**
  - Using the same method name for related tasks makes the code more organized and intuitive.
- **Provides flexibility in method calls**
  - The programmer can call the same method with different types or numbers of arguments.

```
// Demonstrate method overloading.
class OverloadDemo {
  void test() {
    System.out.println("No parameters");
  }

  // Overload test for one integer parameter.
  void test(int a) {
    System.out.println("a: " + a);
  }
  // Overload test for two integer parameters.
  void test(int a, int b) {
    System.out.println("a and b: " + a + " " + b);
  }

  // overload test for a double parameter
  double test(double a) {
    System.out.println("double a: " + a);
    return a*a;
  }
}

class Overload {
  public static void main(String args[]) {
    OverloadDemo ob = new OverloadDemo();
    double result;

    // call all versions of test()
    ob.test();
    ob.test(10);
    ob.test(10, 20);
    result = ob.test(123.25);
    System.out.println("Result of ob.test(123.25): " + result);
  }
}
```

OUTPUT

```
No parameters
a: 10
a and b: 10 20
double a: 123.25
Result of ob.test(123.25): 15190.5625
```

HOMEWORK: (Both these programs shared as a separate PDF document)

1. Write a java program to calculate the area of different shapes namely circle,rectangle and triangle using the concept of method overloading.(S4 July 2017 KTU)
2. Explain briefly about method overloading with an example. (S4 May 2019 KTU)

(Solutions to both these questions shared as a separate PDF document.Do refer.)

```
import java.util.Scanner;
class Area {
    // Method to calculate area of a circle
    void calculateArea(double radius) {
        double area = Math.PI * radius * radius;
        System.out.println("Area of Circle = " + area);    }
    // Method to calculate area of a rectangle
    void calculateArea(double length, double width) {
        double area = length * width;
        System.out.println("Area of Rectangle = " + area);}
    // Method to calculate area of a triangle
    void calculateArea(double base, double height, int
triangle) {
        double area = 0.5 * base * height;
        System.out.println("Area of Triangle = " + area);}}
public class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
Area obj = new Area();
System.out.print("Enter the radius of the circle: ");
double radius = sc.nextDouble();
obj.calculateArea(radius);
System.out.print("\nEnter the length of the rectangle: ");
double length = sc.nextDouble();
System.out.print("Enter the width of the rectangle: ");
double width = sc.nextDouble();
obj.calculateArea(length, width);
System.out.print("\nEnter the base of the triangle: ");
double base = sc.nextDouble();
System.out.print("Enter the height of the triangle: ");
double height = sc.nextDouble();
obj.calculateArea(base, height, 1); // Third argument differentiates
the overloaded method}}
```

OVERLOADING CONSTRUCTORS

- In addition to overloading normal methods, constructor methods can also be overloaded.
- Consider the following example;

```
class Box {
  double width;
  double height;
  double depth;

  // This is the constructor for Box.
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

- Here, the Box( ) constructor requires three parameters.
- ie, all the declarations of Box objects must pass three arguments to the Box( ) constructor.
- Eg:

  The following statement is currently invalid:

  Box ob = new Box();
- Since Box( ) requires three arguments, it's an error to call it without them.

- What if we simply wanted a box and did not care what its initial dimensions were?
- What if we want to initialize a cube by specifying only one value that would be used for all three dimensions?
- As the Box class is currently written, these other options are not available.
- Solution => **Constructor Overloading**
- Constructor overloading is the process of **defining two or more constructors in the same class with different parameter lists** (different number, type, or order of parameters).
- **It allows objects to be initialized in different ways.**

Eg1:

```
/* Here, Box defines three constructors to initialize
   the dimensions of a box various ways.
*/
class Box {
  double width;
  double height;
  double depth;

  // constructor used when all dimensions specified
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }

  // constructor used when no dimensions specified
  Box() {
    width = -1;  // use -1 to indicate
    height = -1; // an uninitialized
    depth = -1;  // box
  }

  // constructor used when cube is created
  Box(double len) {
    width = height = depth = len;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class OverloadCons {
  public static void main(String args[]) {
    // create boxes using the various constructors
    Box mybox1 = new Box(10, 20, 15);
    Box mybox2 = new Box();
    Box mycube = new Box(7);

    double vol;
    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume of mybox1 is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume of mybox2 is " + vol);
    // get volume of cube
    vol = mycube.volume();
    System.out.println("Volume of mycube is " + vol);
  }
}
```

OUTPUT:

```
Volume of mybox1 is 3000.0
Volume of mybox2 is -1.0
Volume of mycube is 343.0
```

- The proper overloaded constructor is called based upon the parameters specified when new is executed.

HOMEWORK:

1. Write a java program with a class ‘Complex’ to represent complex numbers. Model the class in such a way that it uses constructor overloading aspects to initialize it’s instances.(S4 Dec 2019 KTU)

Method Overriding: (Run-Time Polymorphism)

- Method overloading has the **same name for the methods used in the superclass as well as subclass.**
- The **method used in the subclass overrides the definition of the same method used in the superclass.**
- It is **run-time polymorphism.**

Rules for method overriding:

- The method in the subclass and superclass must have the **same name.**
- They must have the **same parameter list.**
- They must have the **same return type or a covariant return type.**
- If the method name and parameter list are the same in the superclass and the inherited subclass, then the subclass method overrides the definition of superclass method => METHOD OVERRIDING
- Inheritance is essential for performing overriding.
- Private, static, and final methods cannot be overridden.
- **@Override** is an optional field, which is used before the definition of the overriding method for understanding purpose.

Advantages of Method Overriding:

- Supports **run-time polymorphism** (dynamic method dispatch).
- Allows subclasses to provide specific implementations.
- Improves **code reusability and flexibility.**
- **Makes programs easier to maintain and extend.**

Eg: Program to demonstrate method overriding

```
class BaseClass {
    public void myFunction() {
        System.out.println("Base Class is executed");
    }
}

class DerivedClass extends BaseClass {
    @Override
    public void myFunction() {
        System.out.println("Derived Class is executed");
    }
}

public class OverrideMain {
    public static void main(String[] args) {
        DerivedClass obj1 = new DerivedClass();
        obj1.myFunction();       // calls Derived Class method
    }
}
```

OUTPUT:

Derived class is executed.

- Here,
- Parent class : BaseClass
- Child class: DerivedClass

- Method myFunction( ) is defined in both parent and child classes.
- Method myFunction( ) have same parameters and return type as well.
- So, the DerivedClass method overrides the definition of the BaseClass method.

METHOD OVERLOADING V/S METHOD OVERRIDING

METHOD OVERLOADING vs METHOD OVERRIDING

	Feature	Method Overloading	Method Overriding
1.	Definition	Same method name with different parameter list within the same class.	Same method name with same parameter list in superclass and subclass.
2.	Parameter List	Must be different in number, type or order of parameters.	Must be exactly the same.
3.	Return Type	May be same or different.	Must be same or covariant return type.
4.	Inheritance	Not required.	Required (IS-A relationship).
5.	Polymorphism	Compile-time (Static) polymorphism.	Run-time (Dynamic) polymorphism.
6.	Method Binding	Early Binding (at compile time).	Late Binding (at run time).
7.	Which Method is Called?	Determined by reference type at compile time.	Determined by object type at run time.
8.	Where it Occurs?	Within the same class.	In superclass and subclass.
9.	Access Modifier	No special restriction.	Cannot reduce the visibility, (Cannot be more restrictive)
10.	Restrictions	None.	final, static and private methods cannot be overridden.
11.	Example	`class Demo { void sum(int a, int b) { } void sum(int a, int b, int c) { } }`	`class A { void show() { } } class B extends A { @Override void show() { } }`

DYNAMIC METHOD DISPATCH:

- Dynamic method dispatch is **one of Java's most powerful concepts.**
- Method overriding forms the basis for this.
- Dynamic method dispatch is the **mechanism by which a call to an overridden method is resolved at run time**, rather than compile time.
- Java implements run-time polymorphism with the help of dynamic method dispatch.

Eg:

Dynamic Method Dispatch – Java Program

```
class BaseClass {
    public void myFunction() {
        System.out.println("Base Class is executed");
    }
}

class DerivedClass extends BaseClass {
    @Override
    public void myFunction() {
        System.out.println("Derived Class is executed");
    }
}

public class OverrideMain {
    public static void main(String[] args) {
        BaseClass obj1 = new DerivedClass(); // upcasting
        obj1.myFunction();                   // calls Derived Class method
    }
}
```

OUTPUT:

Derived class is executed

Example - Explained:

- Name of the superclass: BaseClass.
  - It contains a method: myFunction( )
- Name of the sub class: DerivedClass.
  - It also contains a method: myFunction( )
- Here, the method named myFunction( ) of both the superclass and subclass are having the same name and signature. => METHOD OVERRIDING
- But, consider the following in the main class

BaseClass obj1 = new DerivedClass(); // This is the key statement for Dynamic Method Dispatch.

- It has two different types:
  - Reference variable - obj1 - Of the type BaseClass
  - Actual object created - Of the type DerivedClass
- BaseClass obj1 = new DerivedClass();

↑ ↑

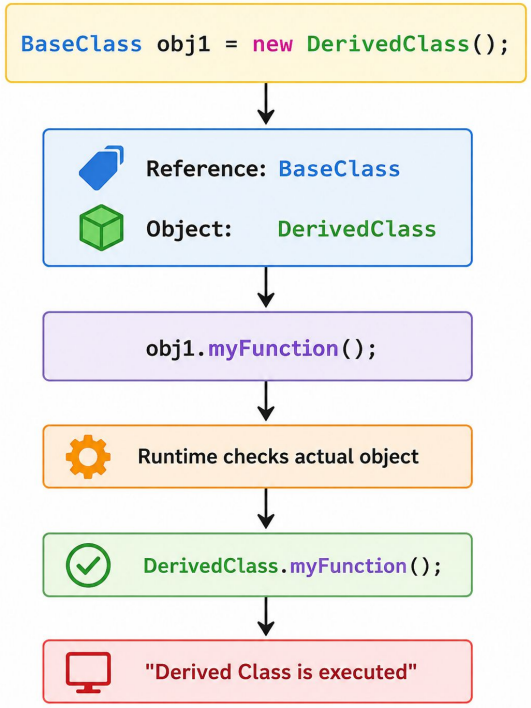
Reference Object

- This is called **upcasting** because a subclass object is assigned to a superclass reference.

- Now, consider the method call;

  obj1.myFunction();

- We may think, obj1 is a BaseClass reference, so the BaseClass method should execute.
- But it doesn't.
- The actual object is: new DerivedClass()
- Therefore, **at runtime, Java determines that the object is a DerivedClass object and calls the overridden method in DerivedClass.**
- So the output is:
  - Derived Class is executed



- In this program,

  BaseClass reference + DerivedClass object → DerivedClass's overridden method is executed.

Why is this called Dynamic Method Dispatch?

- Because, **the decision about which overridden method to execute is made at runtime, based on the actual object, not just the reference type.**
- **In short;**
  - **Reference type decides what methods are accessible, but the actual object type decides which overridden method is executed at runtime.**

RECURSION:

- **Java supports recursion.**
- Recursion is the process of defining something in terms of itself.
- As it relates to Java programming, **recursion is the attribute that allows a method to call itself.**
- A method that calls itself is said to be recursive.
- Eg:
  - The classic example of recursion is the computation of the factorial of a number.
  - The factorial of a number N is the product of all the whole numbers between 1 and N.
  - Eg: 3! = 3*2*1 = 6

Eg:

```
// A simple example of recursion.
class Factorial {
  // this is a recursive method
  int fact(int n) {
    int result;

    if(n==1) return 1;
    result = fact(n-1) * n;
    return result;

  }
}

class Recursion {
  public static void main(String args[]) {
    Factorial f = new Factorial();

    System.out.println("Factorial of 3 is " + f.fact(3));
    System.out.println("Factorial of 4 is " + f.fact(4));
    System.out.println("Factorial of 5 is " + f.fact(5));
  }
}
```

OUTPUT:

```
Factorial of 3 is 6
Factorial of 4 is 24
Factorial of 5 is 120
```

INHERITANCE

- Inheritance in Java is an Object-Oriented Programming (OOP) concept where **one class acquires the properties (fields) and behaviors (methods) of another class**.
- **OOP allows us to derive new classes from existing classes.This is called Inheritance.**
- It is a powerful feature in Java for code reuse, improved maintainability, and represents an "is-a" relationship.
- In Java, inheritance is implemented using the ***extends*** keyword.

Superclass and Subclass:

- OOP allows us to **derive new classes from existing classes**.
- This is called Inheritance.
- **Superclass:**
  - The existing class from which a new class is derived is called the superclass.
  - It is also called the parent class or base class.
- **Subclass:**
  - The new class that is derived from an existing class is called the subclass.
  - It is also called the child class or derived class.
- The general form of a class declaration that inherits a superclass is:

```
class subclass-name extends superclass-name {
 // body of class
}
```

Eg:

```
class Animal {                  // Superclass
    void eat() {
        System.out.println("Animal eats");
    }
}
class Dog extends Animal {      // Subclass
    void bark() {
        System.out.println("Dog barks");
    }
}
```

Here:

- *Animal* is the superclass because it is the existing class.
- *Dog* is the subclass because it is derived from *Animal.*
- extends is the keyword used to create the inheritance relationship.
- *Dog* acquires the *eat()* method from *Animal.*

Animal
Superclass
|
extends
↓
Dog
Subclass

TYPES OF INHERITANCE:

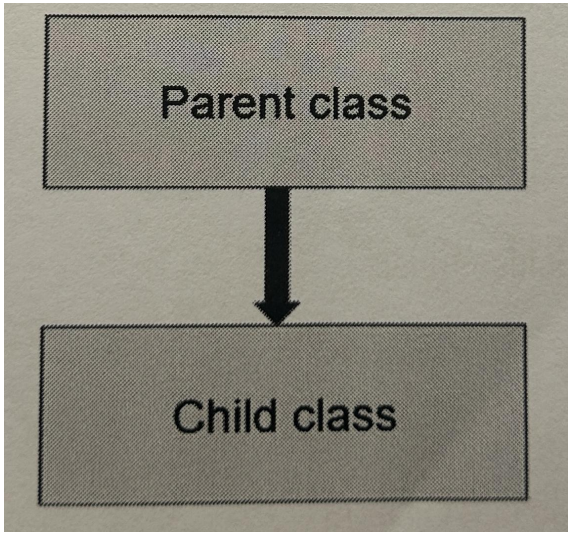
- In general, there are five distinct types of inheritance:
  - **Single Inheritance**
  - **Multilevel Inheritance**
  - **Hierarchical Inheritance**
  - **Multiple Inheritance**
    - Java does not support multiple inheritance using classes because of the Diamond Problem (ambiguity issue).
    - Multiple inheritance can be achieved through interfaces.
  - **Hybrid Inheritance**
    - Combines multiple types of inheritance.
    - Due to ambiguity issues, interfaces are required in Java for such implementations.

Single Inheritance:

- A child class inherits properties, such as instance variables and methods, from only one parent class.
- This is called single inheritance.

Syntax:

```
class parentclass
{
   // execution statements
}
class childclass extends parentclass
{
   // execution statements
}
```



Eg: Single Inheritance

```
1  class Fish
2  {
3      void swim()
4      {
5          System.out.println("Fish Swim in water");
6      }
7  }
8
9  class Goldfish extends Fish
10 {
11     void color()
12     {
13         System.out.println("Golden Color");
14     }
15 }
16
17 class SingleInheritanceMain
18 {
19     public static void main(String[] args)
20     {
21         Goldfish goldfishobj = new Goldfish();
23         goldfishobj.color();
           goldfishobj.swim();
       }
   }
```

OUTPUT:

Golden Color
Fish Swim in water

Here,

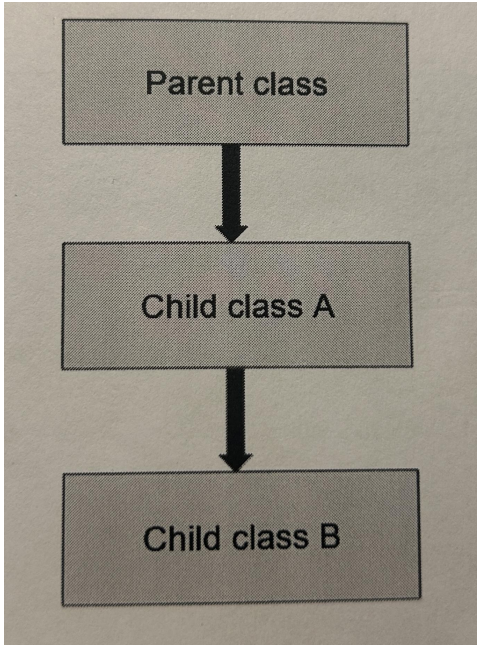
- Fish is the parent class.
- Goldfish is the child class.
- Goldfish extends Fish means Goldfish inherits the methods of Fish.
- color() belongs to Goldfish.
- swim() belongs to Fish, but can be called using the Goldfish object because it is inherited.

Multilevel Inheritance

- A child class (childclassB) inherits properties such as instance variables and methods from another child class (childclassA) that itself is inherited from a parent class.
- This is called multilevel inheritance.

Syntax:

```
class parentclass
{
 // execution statements
}
class childclassA extends parentclass
{
   // execution statements
}
class childclassB extends childclassA
{
   // execution statements
}
```



```java
class Animal
{
    void play()
    {
        System.out.println("Fish plays inside water");
    }
}

class Fish extends Animal
{
    void swim()
    {
        System.out.println("Fish Swim in water");
    }
}

class Goldfish extends Fish
{
    void color()
    {
        System.out.println("Golden Color");
    }
}
class MultilevelInheriMain
{
    public static void main(String[] args)
    {
        Goldfish goldfishobj = new Goldfish();

        goldfishobj.color();
        goldfishobj.swim();
    }   goldfishobj.play();
}
```

OUTPUT:

Golden Color
Fish Swim in water
Fish plays inside water.

Note:

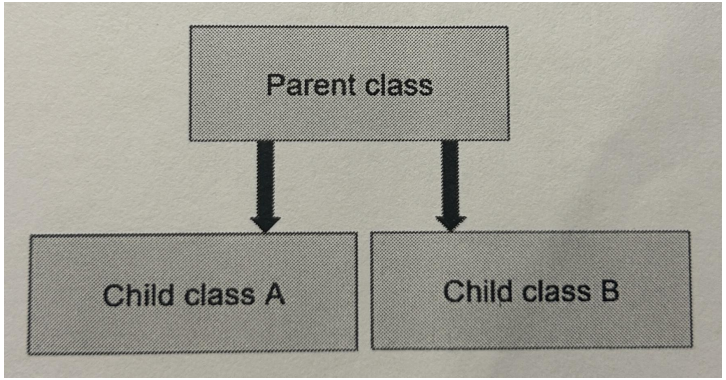
- *Goldfish extends Fish.*
- *Fish extends Animal.*
- *So, a Goldfish object can access methods from both Fish and Animal.*

Hierarchical Inheritance:

- Multiple child classes (Child class A and Child class B) inherit properties, such as instance variables and methods, from only one parent class.
- This is called hierarchical inheritance.

Syntax:

```
class parentclass
{
   // execution statements
}
class childclassA extends parentclass
{
   // execution statements
}
class childclassB extends parentclass
{
   // execution statements
}
```



Hierarchical Inheritance – Java Program

```
class Animal
{
    void sound()
    {
        System.out.println("Animals make sound");
    }
}

class Cat extends Animal
{
    void color()
    {
        System.out.println("black color");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("Dog barks");
    }
}

class hierInheriMain
{
    public static void main(String[] args)
    {
        Cat catobj = new Cat();
        Dog dogobj = new Dog();

        catobj.color();
        dogobj.bark();
        catobj.sound();
        dogobj.sound();
    }
}
```

OUTPUT:

black color

Dog barks

Animals make sound

Animals make sound

Note:

- *Animal is the common parent class.*
- *Cat and Dog are separate child classes.*
- *Both Cat and Dog inherit sound() from Animal.*
- *Cat has its own color() method.*
- *Dog has its own bark() method.*

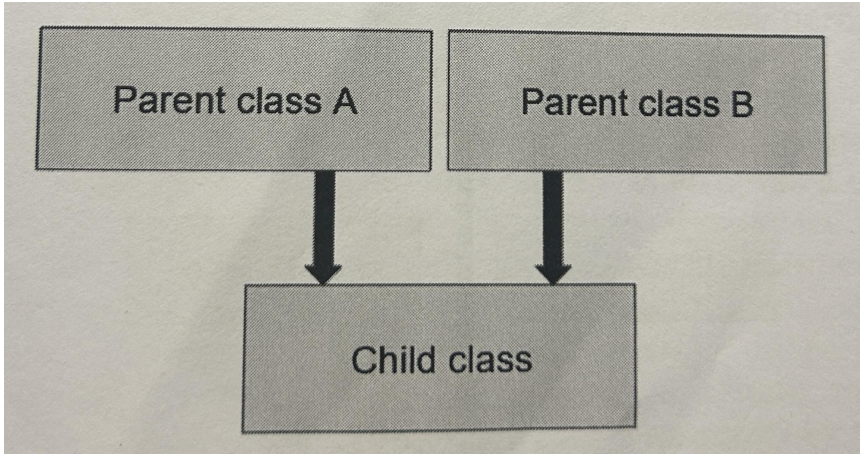
HOMEWORK:

1. Write a Java program that creates a class named 'Employee' having the following members: Name, Age, Phone number, Address, and Salary. It also has a method named printSalary() which prints the salary of the Employee. Two classes, 'Officer' and 'Manager' inherits the 'Employee' class. The 'Officer' and 'Manager' classes have data members 'specialization' and department,respectively. Now, assign name, age, phone number, address, and salary to an officer and a manager by making objects of both of these classes, and print the same.

(Solution shared as separate document)

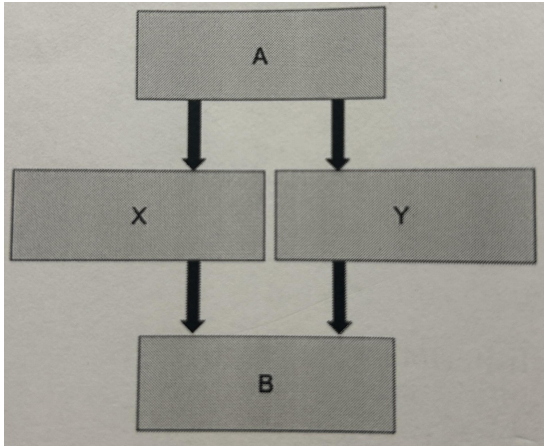
Multiple Inheritance:

- A child class inherits properties, such as instance variables and methods, from more than one parent class (Parent class A and Parent class B).
- This is called multiple inheritance.
- **Multiple inheritance is not supported in Java** using classes because it can create the **Diamond Problem**, which causes ambiguity.



Diamond Problem:

- Consider the given structure.
- Here, A is the common parent class.
- X and Y inherit from A.
- B attempts to inherit from both X and Y.
- Suppose class A contains a method:

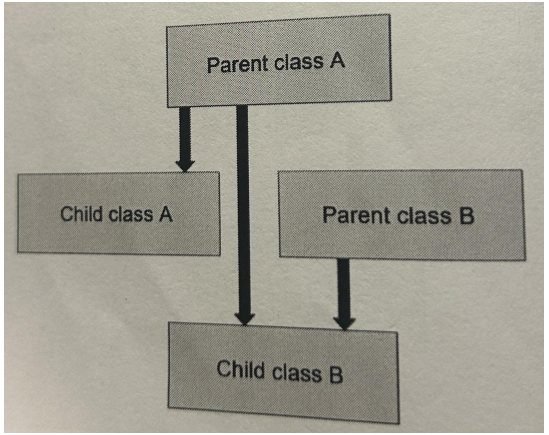



```
void display()
{
    System.out.println("Class A");
}
```

- If both X and Y inherit this method and B inherits from both X and Y, then there can be ambiguity about which version of display() should be executed.
- This ambiguity is known as the Diamond Problem.
- Java avoids this ambiguity by not allowing multiple inheritance through classes.
- However, multiple inheritance can be achieved through interfaces.

Hybrid Inheritance:

- Hybrid inheritance **combines multiple types of inheritance.**
- It is a **combination of two or more types of inheritance**, such as:
  - Single inheritance
  - Multilevel inheritance
  - Hierarchical inheritance
  - Multiple inheritance



- Because hybrid inheritance may involve multiple inheritance, it can lead to the Diamond Problem and ambiguity.
- Therefore, interfaces are used in Java to implement such inheritance structures without the ambiguity associated with multiple class inheritance.